Bioinf Pipelines. HW2

Deadline: 22.04
Performed by Victor Florin

You have to implement a pipeline that:

1. Takes local fastq reads or downloads them from NCBI by ID (depending on the input params)
2. Runs QC on the raw reads
3. Trims the reads
4. Runs QC on the trimmed reads
5. Assembles them into a de novo reference or takes an input reference file (depending on the input params)
6. Maps the trimmed reads to the reference
7. Plots coverage

15 points if steps 1-4 are completed; 30 points if all steps are completed.
Advice: do not use large files for testing, consider E.Coli or some small virus. For example, you can use this HIV reads: DRR030302.

1. Preparing the environment and writing scripts

I chose SRR1175163.

- Organism: ***Dengue virus***
- Size: 9.4Mb

Library:

- Name: M105
- Instrument: Illumina MiSeq
- Strategy: WGS
- Source: GENOMIC
- Selection: RANDOM PCR
- Layout: **PAIRED**

Creating & activating the HW2 env:

```
conda create -n hw2 -y
```

```
conda activate hw2
```

Adding channels:

```
conda config --add channels bioconda
conda config --add channels conda-forge
```

Installing needed tools:

```
conda install -y fastqc trimmomatic bwa samtools sra-tools spades
```

Just in case:

```
fastqc --version
trimmomatic -version
bwa 2>&1 | grep "Version"
samtools --version
spades.py --version
```

```
(hw2) student@LAPTOP-NKRTACMG:~$ fastqc --version
FastQC v0.12.1
(hw2) student@LAPTOP-NKRTACMG:~$ trimmomatic -version
0.40
(hw2) student@LAPTOP-NKRTACMG:~$ bwa 2>&1 | grep "Version"
Version: 0.7.19-r1273
(hw2) student@LAPTOP-NKRTACMG:~$ ^C
(hw2) student@LAPTOP-NKRTACMG:~$ samtools --version
samtools 1.23.1
Using htslib 1.23.1
(hw2) student@LAPTOP-NKRTACMG:~$ spades.py --version
SPAdes genome assembler v4.2.0
```

Making structure - creating the project folder and filling it:

```
mkdir hw2_pipeline
cd hw2_pipeline
touch main.nf
touch nextflow.config
mkdir data
```

For nextflow.config:

```
cat > nextflow.config << 'EOF'
params {
    reads = null
```

```
    sra_id = null
    reference = null
}

process {
    executor = 'local'
}
EOF

# cat nextflow.config
```

```
(hw2) student@LAPTOP-NKRTACMG:~/hw2_pipeline$ cat nextflow.config
params {
    reads = null
    sra_id = null
    reference = null
}

process {
    executor = 'local'
}
(hw2) student@LAPTOP-NKRTACMG:~/hw2_pipeline$
```

For main.nf:

```
cat > main.nf << 'EOF'
params.reads = null
params.sra_id = null
params.reference = null

// ========================
// PROCESS: FETCH READS
// ========================
process fetch_reads {
    input:
        val sra_id

    output:
        tuple val(sra_id), path("${sra_id}_1.fastq"), path("${sra_id}_2.fastq")

    script:
    """
    fasterq-dump ${sra_id}
    """
}

// ========================
// PROCESS: QC
```

```
// ========================
process run_qc_raw {
    input:
        tuple val(id), path(r1), path(r2)

    output:
        path "raw_qc/", type: 'dir'

    script:
    """
    mkdir raw_qc
    fastqc -o raw_qc $r1 $r2
    """
}

process run_qc_trimmed {
    input:
        tuple val(id), path(r1), path(r2)

    output:
        path "trimmed_qc/", type: 'dir'

    script:
    """
    mkdir trimmed_qc
    fastqc -o trimmed_qc $r1 $r2
    """
}
// ========================
// PROCESS: TRIMMING
// ========================
process trimm {
    input:
        tuple val(id), path(r1), path(r2)

    output:
        tuple val(id), path("R1_p.fq"), path("R2_p.fq")

    script:
    """
    trimmomatic PE \
        $r1 $r2 \
        R1_p.fq R1_u.fq \
        R2_p.fq R2_u.fq \
        LEADING:3 TRAILING:3 MINLEN:36
    """
```

```
}

// ========================
// PROCESS: ASSEMBLY
// ========================
process assembly {
    input:
        tuple val(id), path(r1), path(r2)

    output:
        path "assembly/contigs.fasta"

    script:
    """
    spades.py -1 $r1 -2 $r2 -o assembly
    """
}

// ========================
// PROCESS: MAPPING
// ========================
process mapping {
    input:
        tuple val(id), path(r1), path(r2)
        path ref

    output:
        path "aligned.bam"

    script:
    """
    bwa index $ref
    bwa mem $ref $r1 $r2 | samtools view -bS - | samtools sort -o aligned.bam
    """
}

// ========================
// PROCESS: COVERAGE
// ========================
process coverage {
    input:
        path bam

    output:
        path "coverage.txt"
```

```
    script:
    """
    samtools depth $bam > coverage.txt
    """
}

// =======================
// WORKFLOW
// =======================
workflow pipeline {

    take:
        reads_ch

    main:
        qc1 = run_qc_raw(reads_ch)

        trimmed = trimm(reads_ch)

        qc2 = run_qc_trimmed(trimmed)

        if (params.reference) {
            ref = channel.of(file(params.reference))
        } else {
            ref = assembly(trimmed).map{ it }
        }

        mapped = mapping(trimmed, ref)

        cov = coverage(mapped)

    emit:
        qc1 = qc1
        qc2 = qc2
        coverage = cov
}

// =======================
// ENTRY WORKFLOW
// =======================
workflow {

    main:

    if (params.sra_id) {
        reads = fetch_reads(params.sra_id)
```

```
    } else {
        reads = channel.fromFilePairs(params.reads)
    }

    pipeline(reads)

    publish:
        qc1 = pipeline.out.qc1
        qc2 = pipeline.out.qc2
        coverage = pipeline.out.coverage
}

// ========================
// OUTPUT
// ========================
output {

    qc1 {
        path 'qc_initial'
    }

    qc2 {
        path 'qc_trimmed'
    }

    coverage {
        path 'coverage'
    }
}
EOF

# cat main.nf | head -20 ; tail -20 main.nf ; wc -l main.nf
```

```
(hw2) student@LAPTOP-NKRTACMG:~/hw2_pipeline$ cat main.nf | head -20
params.reads = null
params.sra_id = null
params.reference = null

// ========================
// PROCESS: FETCH READS
// ========================
process fetch_reads {
    input:
        val sra_id

    output:
        tuple val(sra_id), path("${sra_id}_1.fastq"), path("${sra_id}_2.fastq")

    script:
    """
    fasterq-dump ${sra_id}
    """
}
```

Running the pipeline on **SRR1175163**:

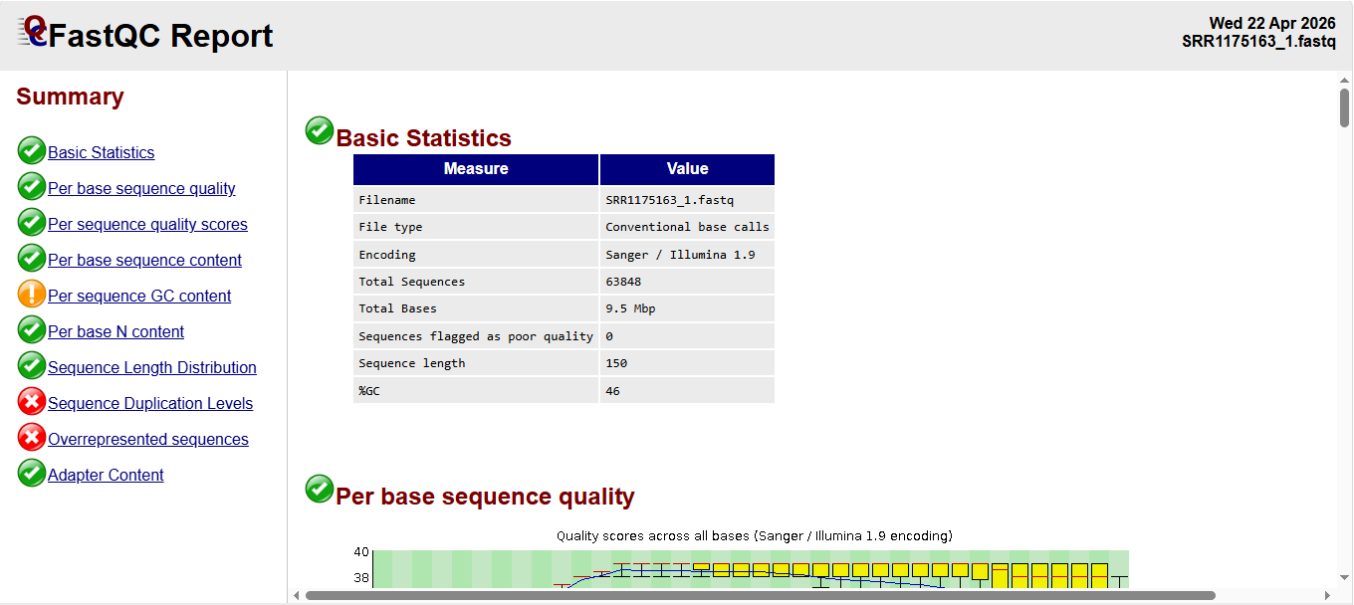
```
nextflow run main.nf --sra_id SRR1175163
```

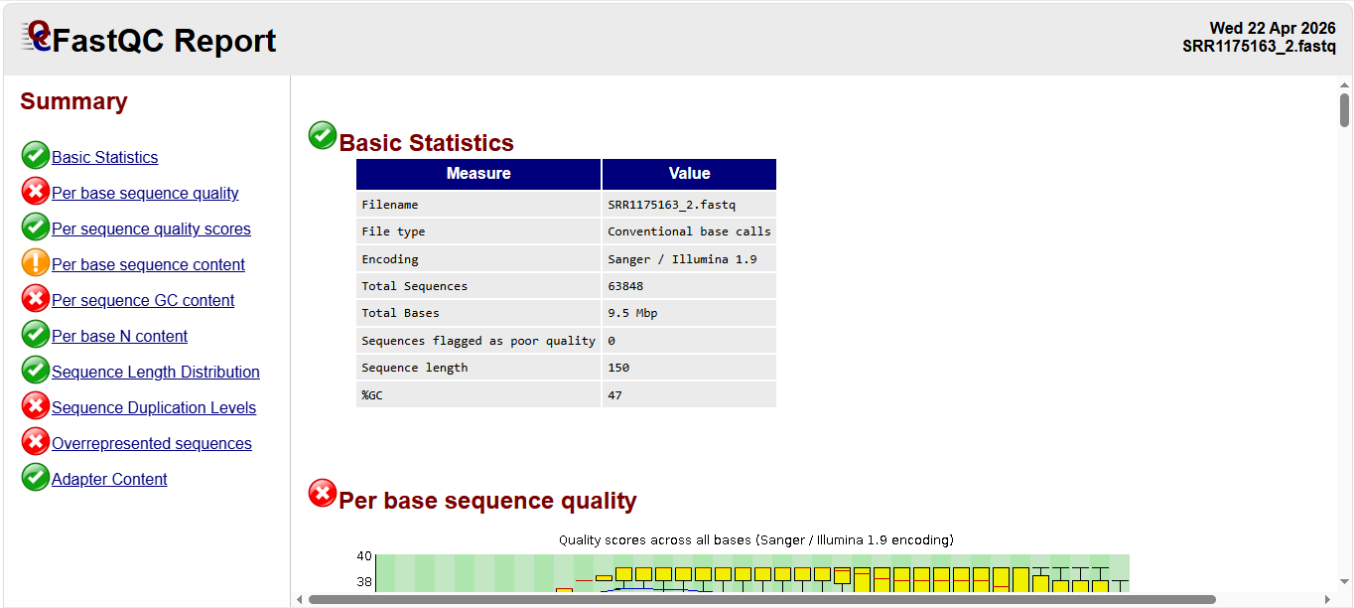
```
(hw2) student@LAPTOP-NKRTACMG:~/hw2_pipeline$ nextflow run main.nf --sra_id SRR1175163

 N E X T F L O W  ~  version 25.10.4

Launching `main.nf` [elated_bohr] DSL2 - revision: 8dc21e5125

executor >  local (7)
[cf/573dbf] fetch_reads             | 1 of 1 ✔
[72/58bd78] pipeline:run_qc_raw     | 1 of 1 ✔
[02/7c5613] pipeline:trimm          | 1 of 1 ✔
[d6/bee9bb] pipeline:run_qc_trimmed | 1 of 1 ✔
[57/8b0b2f] pipeline:assembly       | 1 of 1 ✔
[86/984089] pipeline:mapping        | 1 of 1 ✔
[7e/483f5e] pipeline:coverage       | 1 of 1 ✔
Completed at: 22-Apr-2026 13:54:05
Duration    : 1m 52s
CPU hours   : (a few seconds)
Succeeded   : 7
```

---

2. QC on raw data

Done. See **SRR1175163_1_fastqc.html** and **SRR1175163_2_fastqc.html** in HW2/hw2_pipeline/work/72/58bd7820875334c55ce5b16e87927f/raw_qc





---

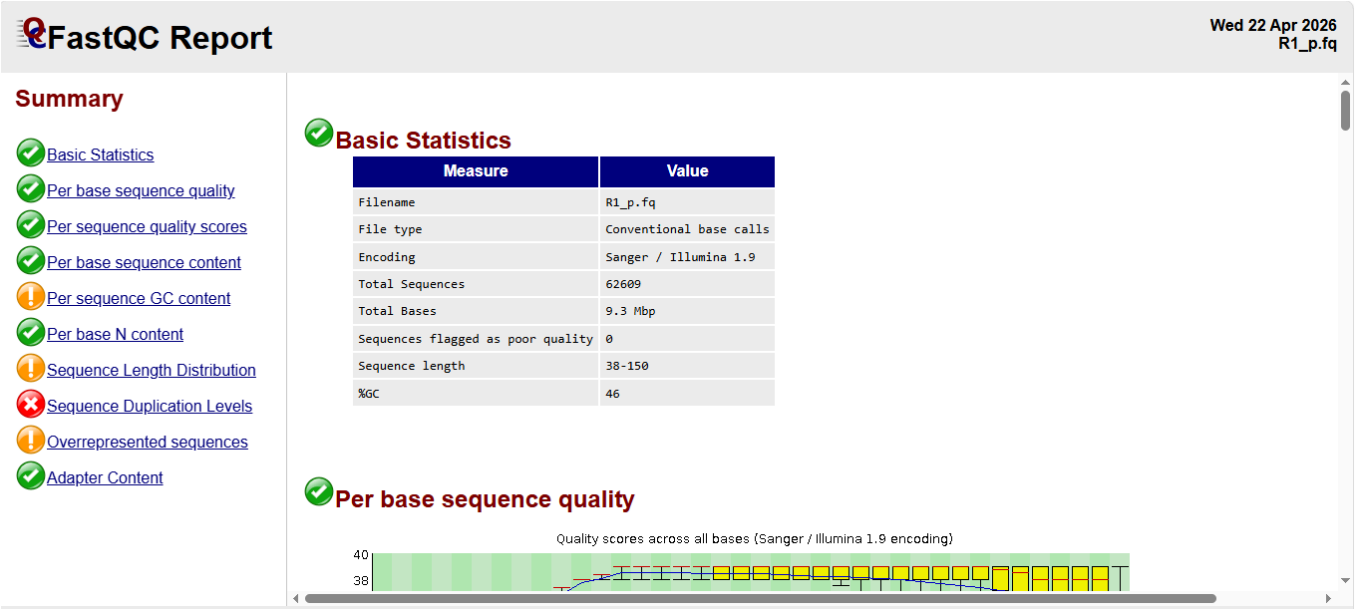
3. Trimming

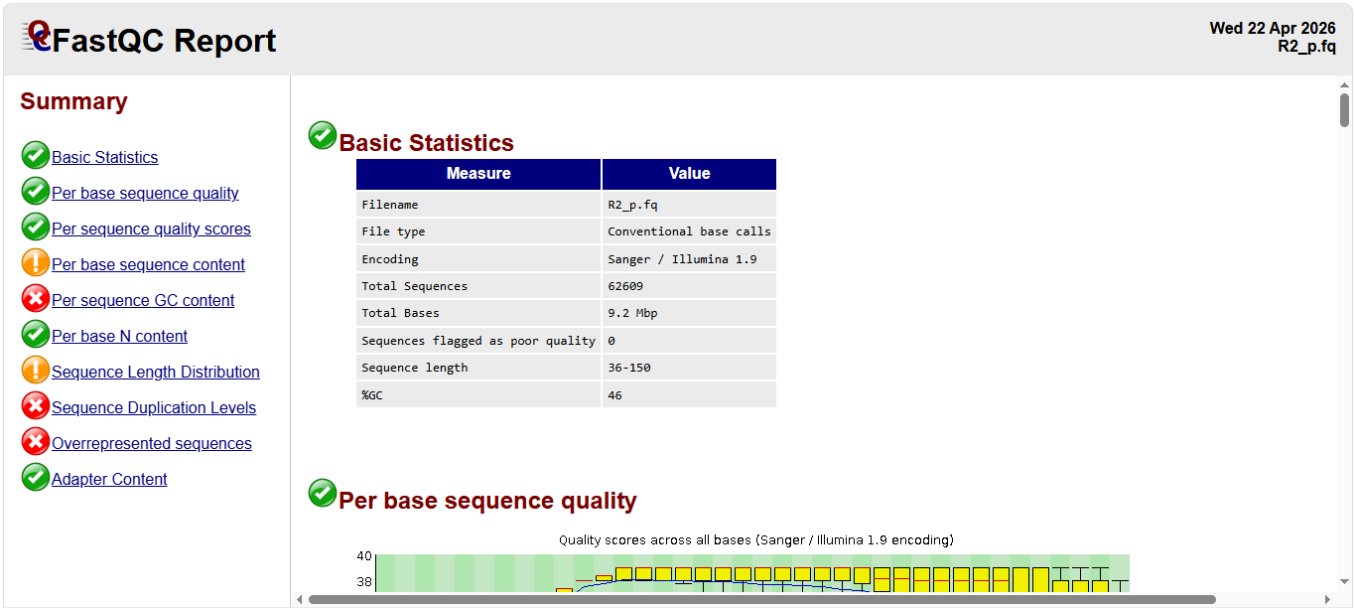
Done. See HW2/hw2_pipeline/work/02/7c561346ef7306aa4521c85b0f22fe.

---

4. QC on trimmed data

Done. See **R1_p_fastqc.html** и **R2_p_fastqc.html**
HW2/hw2_pipeline/work/d6/bee9bbc21dd3a452c7386ca47717f2/trimmed_qc





---

5. Assembly

Done. See HW2/hw2_pipeline/work/57/8b0b2f420b2464c751609e932597ac.

---

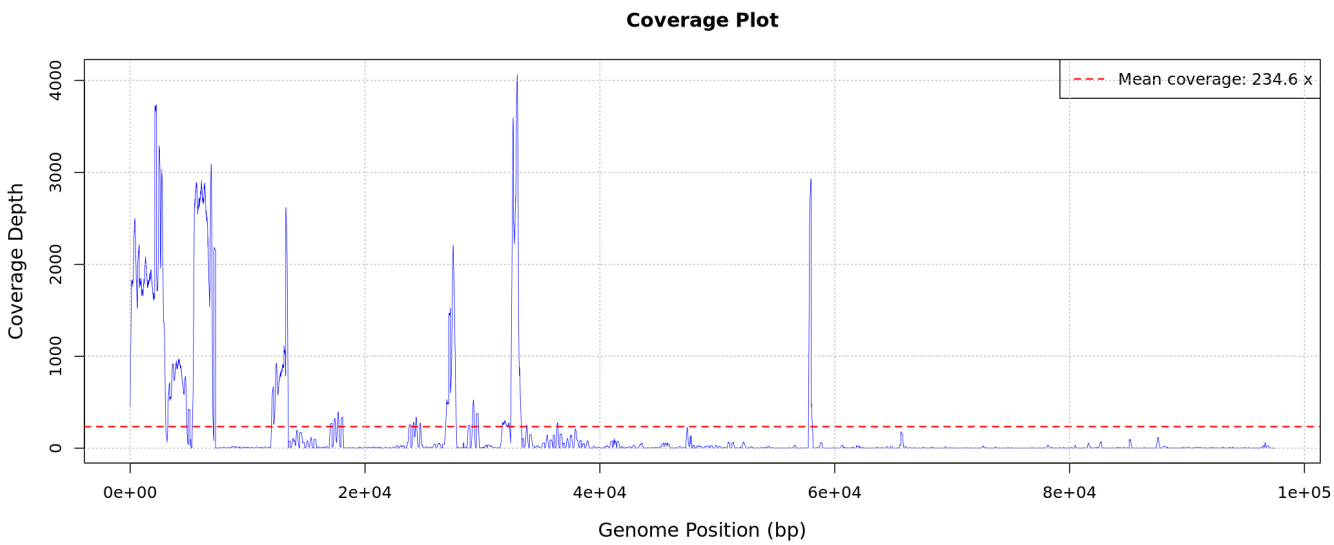
6. Mapping

Done. See HW2/hw2_pipeline/work/7e/483f5e4aee835390318a040181f55d.

---

7. Coverage plot

Let's take the fuction of coverage depth on genomic position.

```
cat > plot_coverage.R << 'EOF'
cat("Loading coverage.txt...\n")
data <- read.table("coverage.txt", header=FALSE, col.names=c("contig", "pos",
"cov"))

data$cumulative_pos <- 0
current_offset <- 0
last_contig <- data$contig[1]

for (i in 1:nrow(data)) {
  if (data$contig[i] != last_contig) {
    current_offset <- current_offset + max(data$pos[data$contig == last_contig]) +
100
    last_contig <- data$contig[i]
  }
  data$cumulative_pos[i] <- current_offset + data$pos[i]
}

cat("Loaded", nrow(data), "positions\n")
cat("Number of contigs:", length(unique(data$contig)), "\n")

png("coverage_r.png", width=14, height=6, units="in", res=150)

plot(data$cumulative_pos, data$cov,
     type="l",
     col="blue",
     lwd=0.5,
     xlab="Genome Position (bp)",
     ylab="Coverage Depth",
     main="Coverage Plot",
     cex.lab=1.2)

avg_cov <- mean(data$cov)
abline(h=avg_cov, col="red", lty=2, lwd=1.5)

legend("topright",
       legend=c(paste("Mean coverage:", round(avg_cov, 1), "x")),
       col="red", lty=2, lwd=1.5)

grid(col="gray", lty=3)

dev.off()

cat("Saved: coverage_r.png\n")
```

```
cat(paste("\nStatistics:\n"))
cat(paste("  Mean coverage:", round(avg_cov, 1), "x\n"))
cat(paste("  Min coverage:", min(data$cov), "x\n"))
cat(paste("  Max coverage:", max(data$cov), "x\n"))
cat(paste("  Total positions:", format(nrow(data), big.mark=","), "\n"))
cat(paste("  Number of contigs:", length(unique(data$contig)), "\n"))
EOF

Rscript plot_coverage.R
# rm -rf .nextflow/ work/ results/
# rm -f .nextflow.log*
# cp -r /home/student/hw2_pipeline
/mnt/c/Users/Student/Documents/GitHub/Bioinformatics-Pipelines/HW2/
```